



JavaScript

Zero to Advanced — Complete Course

Syntax · Semantics · Patterns · Modern ES6+

Taught the MIT way — precise, deep, and practical

Instructor

Professor of Computer Science · MIT CSAIL

Specialization: Programming Languages & Web Systems

12

Chapters

60+

Topics

150+

Code Examples

ES3→ES2024

Coverage

Table of Contents

Ch 1 JavaScript Foundations

What is JS · Execution model · Script tag · Strict mode · Comments

Ch 2 Variables & Data Types

var / let / const · Primitives · typeof · Type coercion · null vs undefined

Ch 3 Operators & Expressions

Arithmetic · Comparison · Logical · Nullish · Optional chaining · Comma

Ch 4 Control Flow

if/else · switch · Ternary · for · while · do-while

Ch 5 Functions — Deep Dive

Declaration vs Expression · Arrow fns · Parameters · Default · Rest · Spread

Ch 6 Objects & Prototypes

Object literals · Dot vs bracket · Methods · this · Constructor fns · new

Ch 7 Arrays — Complete Guide

Creation · Indexing · Mutation methods · Iteration methods · Searching · Sorting

Ch 8 ES6+ Modern JavaScript

Classes · Modules · Template literals · Symbol · Map · Set

Ch 9 Asynchronous JavaScript

Call stack · Event loop · Callbacks · Promises · async/await · Error handling

Ch 10 Error Handling & Debugging

try/catch/finally · Error types · Custom errors · throw · Debugging techniques

Ch 11 Advanced Patterns & Concepts

Scope · Hoisting · Closures advanced · Currying · Memoization · Design patterns

Ch 12 The JavaScript Runtime

Memory model · Garbage collection · Event loop deep dive · Microtasks vs Macrotasks · Performance tips

CHAPTER 1

JavaScript Foundations

1.1 What Is JavaScript?

JavaScript is a **high-level, interpreted, dynamically typed, single-threaded** programming language originally designed by Brendan Eich at Netscape in 1995. Today it is standardized as **ECMAScript** (ES) by TC39 and runs in every web browser, on servers via Node.js, on mobile devices, IoT hardware, and more.

The three foundational technologies of the web are HTML (structure), CSS (presentation), and JavaScript (behavior). Unlike the other two, JavaScript is a full Turing-complete programming language.

Where JavaScript Runs

- **Browser (client-side):** V8 (Chrome/Edge), SpiderMonkey (Firefox), JavaScriptCore (Safari)
- **Server-side:** Node.js, Deno, Bun — all built on V8
- **Embedded / Edge:** Cloudflare Workers, AWS Lambda@Edge

■ *JavaScript ≠ Java. They share superficial syntax but are entirely different languages with different type systems, memory models, and use cases.*

1.2 How JavaScript Executes

JavaScript is **interpreted at runtime** — modern engines (like V8) use **JIT (Just-In-Time) compilation** to convert JS into optimized machine code on the fly. Execution follows a single-threaded model with an **event loop** (covered in Ch. 12).

The execution process: **Parsing** → **AST generation** → **Compilation** → **Execution**. Any syntax error at parse time halts the entire script.

1.3 Adding JavaScript to a Webpage

Three ways to include JavaScript in an HTML document:

Inline (avoid for large code)

```
<button onclick="alert('Hello!')">Click me</button>
```

Internal Script Tag

```
<script>

console.log("Hello from internal script");

</script>
```

External File (recommended)

```
<!-- Place before </body> or use defer/async -->

<script src="app.js" defer></script>

<!-- defer: executes after HTML parsed, in order -->

<!-- async: executes as soon as downloaded, no order guarantee -->
```

■ *Always prefer external files for maintainability. Use **defer** for most scripts — it never blocks HTML parsing and respects script order.*

1.4 Strict Mode

Introduced in ES5, '**use strict**' opts your code into a restricted variant of JavaScript that catches common mistakes at runtime and disables some legacy features.

```
'use strict'; // At top of file or function

x = 5; // ReferenceError: x is not defined

// Without strict mode, this creates a global variable silently

delete Object.prototype; // TypeError in strict mode
```

- Prevents accidental globals
- Throws on duplicate parameter names
- Disables **with** statement
- **ES Modules** are always in strict mode automatically

1.5 Comments

```
// Single-line comment

/*
Multi-line comment.

Useful for documentation blocks.

*/

/**
```


CHAPTER 2

Variables & Data Types

2.1 Variable Declarations: var, let, const

JavaScript has three keywords for declaring variables. Understanding their differences in scope, hoisting, and mutability is fundamental.

```
// var – function-scoped, hoisted, re-declarable (avoid in modern JS)

var age = 25;

var age = 30; // No error – re-declaration allowed


// let – block-scoped, not re-declarable, mutable

let score = 100;

score = 200; // OK – reassignment allowed

// let score = 300; // SyntaxError – cannot re-declare


// const – block-scoped, not re-declarable, not reassignable

const PI = 3.14159;

// PI = 3; // TypeError – assignment to constant variable


// IMPORTANT: const with objects/arrays – the binding is const, not the value

const user = { name: 'Alice' };

user.name = 'Bob'; // OK – mutating the object is allowed

// user = {}; // TypeError – can't reassign the binding
```

Feature	var	let	const
Scope	Function	Block	Block

Hoisted	Yes (undefined)	Yes (TDZ)	Yes (TDZ)
Re-declarable	Yes	No	No
Reassignable	Yes	Yes	No
Global property	Yes (window.x)	No	No

■ *Rule of thumb: use **const** by default. Switch to **let** only when you need to reassign. Never use **var** in new code.*

2.2 Primitive Data Types

JavaScript has **7 primitive types**. Primitives are immutable and compared by value.

```
// 1. Number – 64-bit IEEE 754 floating point (no separate int type)

let n1 = 42;

let n2 = 3.14;

let n3 = -Infinity;

let n4 = NaN; // 'Not a Number' – result of invalid math

console.log(NaN === NaN); // false! Use Number.isNaN() instead


// 2. BigInt – arbitrary precision integers (ES2020)

let big = 9007199254740991n; // append 'n'

let sum = big + 1n; // Must mix with other BigInts only


// 3. String – immutable sequence of UTF-16 code units

let s1 = 'single quotes';

let s2 = "double quotes";

let s3 = `template literal – interpolate: ${1 + 1}`; // ES6


// 4. Boolean

let t = true;

let f = false;
```

```
// 5. undefined – variable declared but not assigned

let x;

console.log(x); // undefined


// 6. null – intentional absence of value (object-type quirk)

let empty = null;

console.log(typeof null); // 'object' – historic bug in JS


// 7. Symbol – unique, immutable identifier (ES6)

const id1 = Symbol('id');

const id2 = Symbol('id');

console.log(id1 === id2); // false – every Symbol is unique
```

2.3 The Object Type (Non-Primitive)

Everything that is not a primitive is an **object** — including arrays, functions, dates, regex, and plain objects. Objects are compared by reference.

```
const obj = { a: 1 };

const arr = [1, 2, 3];

const fn = function() {};


// Reference comparison

const a = { x: 1 };

const b = { x: 1 };

console.log(a === b); // false – different references

const c = a;

console.log(a === c); // true – same reference
```

2.4 typeof Operator

```
typeof 42 // 'number'

typeof 'hello' // 'string'

typeof true // 'boolean'

typeof undefined // 'undefined'

typeof null // 'object' ← famous bug

typeof {} // 'object'

typeof [] // 'object' ← use Array.isArray() instead

typeof function(){} // 'function'

typeof Symbol() // 'symbol'

typeof 42n // 'bigint'
```

2.5 Type Coercion

JavaScript automatically converts types in certain contexts. This is one of the most misunderstood features of the language.

```
// Implicit coercion – happens automatically

'5' + 3 // '53' (3 coerced to string – + prefers string)

'5' - 3 // 2 (string coerced to number – - is numeric)

true + true // 2 (booleans coerce to 0/1 for math)

null + 1 // 1

undefined + 1 // NaN


// Explicit coercion – always prefer this

Number('42') // 42

Number(true) // 1
```

```
Number(null) // 0

Number(undefined) // NaN

Number('') // 0


String(42) // '42'


Boolean(0) // false

Boolean('') // false

Boolean(null) // false

Boolean(undefined) // false

Boolean(NaN) // false

Boolean({}) // true – empty object is truthy!

Boolean([]) // true – empty array is truthy!
```

■ **Falsy values:** *false, 0, -0, 0n, "", null, undefined, NaN. Everything else is **truthy**.*

2.6 == vs === (Equality)

```
// === (strict equality) – checks value AND type. Always use this.

1 === 1 // true

1 === '1' // false

null === null // true


// == (loose equality) – performs type coercion. Avoid.

1 == '1' // true – string coerced to number

null == undefined // true – special rule

0 == false // true

'' == false // true
```


CHAPTER 3

Operators & Expressions

3.1 Arithmetic Operators

```
+ - * / // addition, subtraction, multiplication, division

% // modulo (remainder): 10 % 3 === 1

** // exponentiation (ES2016): 2 ** 8 === 256

++x x++ // pre/post increment

--x x-- // pre/post decrement

// Pre vs Post increment

let a = 5;

console.log(++a); // 6 – increments THEN returns

console.log(a++); // 6 – returns THEN increments

console.log(a); // 7
```

3.2 Assignment Operators

```
x = 5 // basic assignment

x += 3 // x = x + 3

x -= 3 // x = x - 3

x *= 3 // x = x * 3

x /= 3 // x = x / 3

x %= 3 // x = x % 3

x **= 3 // x = x ** 3
```

```
// Logical assignment (ES2021)

x &&= val // x = x && val – assign only if x is truthy

x ||= val // x = x || val – assign only if x is falsy

x ??= val // x = x ?? val – assign only if x is null/undefined
```

3.3 Comparison Operators

```
=== // strict equal

!== // strict not equal

> // greater than

< // less than

>= // greater than or equal

<= // less than or equal

// String comparison is lexicographic (Unicode order)

'banana' > 'apple' // true

'10' > '9' // false! ('1' < '9' in Unicode)

10 > 9 // true (use numbers for numeric comparison)
```

3.4 Logical Operators

```
&& // AND – returns first falsy value, or last value

|| // OR – returns first truthy value, or last value

! // NOT – converts to boolean and inverts

// Short-circuit evaluation

false && doSomething() // doSomething never called

true || doSomething() // doSomething never called
```

```
// Practical uses

const name = user && user.name; // guard chaining

const lang = userLang || 'en'; // default value

const active = !isDisabled;

// Double negation !! – convert any value to boolean

!!0 // false

!!'hello' // true
```

3.5 Nullish Coalescing (??) and Optional Chaining (?.)

These ES2020 operators are essential for safe property access and default values.

```
// ?? – returns right side only when left is null or undefined

// Unlike ||, it does NOT trigger on 0, '', or false

const count = userCount ?? 0;

0 ?? 'default' // 0 (0 is not null/undefined)

'' ?? 'default' // '' ('' is not null/undefined)

null ?? 'default' // 'default'

// ?. – optional chaining: short-circuits to undefined if null/undefined

const city = user?.address?.city; // safe even if address is null

const first = arr?.[0]; // safe array access

obj?.method?.(); // safe method call

// Combining both

const zip = user?.address?.zip ?? 'N/A';
```

3.6 Bitwise Operators

Bitwise operators work on 32-bit integer representations. Rarely needed in day-to-day JS, but appear in low-level operations, flag systems, and performance-critical code.

```
& // AND : 5 & 3 = 1 (0101 & 0011 = 0001)

| // OR : 5 | 3 = 7 (0101 | 0011 = 0111)

^ // XOR : 5 ^ 3 = 6 (0101 ^ 0011 = 0110)

~ // NOT : ~5 = -6 (inverts bits, ~n = -(n+1))

<< // Left shift : 1 << 3 = 8

>> // Right shift : 8 >> 2 = 2

>>> // Unsigned right shift

// Common use: fast integer truncation (instead of Math.floor for positives)

~~3.7 // 3

3.7|0 // 3
```

3.7 Spread and Rest (...)

```
// Spread – expands an iterable in-place

const a = [1, 2, 3];

const b = [...a, 4, 5]; // [1, 2, 3, 4, 5]

const clone = { ...obj, extra: 1 }; // shallow clone + merge

Math.max(...a); // equivalent to Math.max(1, 2, 3)

// Rest – collects remaining arguments/elements into an array

function sum(...nums) { // nums is an array

return nums.reduce((a, b) => a + b, 0);
```


CHAPTER 4

Control Flow

4.1 if / else if / else

```
const score = 85;

if (score >= 90) {

  console.log('A');

} else if (score >= 80) {

  console.log('B'); // this runs

} else if (score >= 70) {

  console.log('C');

} else {

  console.log('F');

}

// Omitting braces works for single statements – avoid this habit

if (x > 0) console.log('positive');
```

4.2 Ternary Operator

```
// condition ? valueIfTrue : valueIfFalse

const label = age >= 18 ? 'adult' : 'minor';

// Nested ternary – readable only when shallow

const grade = score >= 90 ? 'A'
```

```
: score >= 80 ? 'B'

: score >= 70 ? 'C' : 'F';

// Use for expressions, not statements with side effects
```

4.3 switch Statement

```
const day = 'Monday';

switch (day) {

case 'Saturday':

case 'Sunday':

  console.log('Weekend');

  break; // REQUIRED – prevents fall-through

case 'Monday':

  console.log('Start of week');

  break;

default:

  console.log('Weekday');

}

// switch uses === for comparison

// Fall-through (no break) is intentional in the Sat/Sun case above
```

■ Always add **break** unless you intentionally want fall-through. Always add **default** as a safety net.

4.4 for Loop

```
// Classic for – init; condition; update

for (let i = 0; i < 5; i++) {
```

```

console.log(i); // 0 1 2 3 4

}

// Counting down

for (let i = 10; i >= 0; i--) { ... }

// Multiple variables

for (let i = 0, j = 10; i < j; i++, j--) { ... }

// Infinite loop (use with break)

for (;;) {

  if (done) break;

}

```

4.5 while and do...while

```

// while – checks condition BEFORE each iteration

let i = 0;

while (i < 5) {

  console.log(i);

  i++;

}

// do...while – executes body AT LEAST ONCE, then checks

let input;

do {

  input = prompt('Enter a number greater than 10:');

```

```
} while (Number(input) <= 10);
```

4.6 for...of — Iterate Iterables

```
// for...of works on any iterable: arrays, strings, Maps, Sets, generators

const fruits = ['apple', 'banana', 'cherry'];

for (const fruit of fruits) {

  console.log(fruit); // 'apple', 'banana', 'cherry'

}

// With index using entries()

for (const [index, fruit] of fruits.entries()) {

  console.log(index, fruit);

}

// String iteration (iterates over Unicode code points)

for (const char of 'hello') {

  console.log(char); // 'h' 'e' 'l' 'l' 'o'

}
```

4.7 for...in — Iterate Object Keys

```
const person = { name: 'Alice', age: 30, city: 'Boston' };

for (const key in person) {

  console.log(key, person[key]);

}
```

```

// WARNING: for...in also iterates inherited enumerable properties

// Always guard with hasOwnProperty if using on non-plain objects

for (const key in obj) {

  if (Object.hasOwnProperty(obj, key)) { ... } // ES2022

}

// Prefer Object.keys() / Object.values() / Object.entries() for plain objects

```

4.8 break, continue, and Labels

```

// break – exits the innermost loop or switch

for (let i = 0; i < 10; i++) {

  if (i === 5) break;

  console.log(i); // 0 1 2 3 4

}

// continue – skips the rest of current iteration

for (let i = 0; i < 6; i++) {

  if (i % 2 === 0) continue;

  console.log(i); // 1 3 5

}

// Labels – break/continue outer loops (rare, use sparingly)

outer: for (let i = 0; i < 3; i++) {

  for (let j = 0; j < 3; j++) {

    if (j === 1) break outer; // breaks out of BOTH loops

  }

}

```


CHAPTER 5

Functions — Deep Dive

5.1 Function Declaration vs Expression

```
// Function Declaration – hoisted completely (callable before definition)

greet('Alice'); // Works! Declaration is hoisted.

function greet(name) {

  return `Hello, ${name}!`;

}

// Function Expression – NOT hoisted

// greet2('Bob'); // ReferenceError: Cannot access before initialization

const greet2 = function(name) {

  return `Hello, ${name}!`;

};

// Named function expression – useful for recursion and debugging

const factorial = function fact(n) {

  return n <= 1 ? 1 : n * fact(n - 1); // 'fact' is only in scope here

};
```

5.2 Arrow Functions (ES6)

Arrow functions have a shorter syntax and, crucially, do **not** bind their own **this** — they inherit **this** from the enclosing lexical scope.


```
setInterval(() => {

  this.seconds++; // 'this' is the TimerFixed instance – CORRECT

}, 1000);

}
```

5.3 Parameters

```
// Default parameters (ES6) – used when argument is undefined

function greet(name = 'World', greeting = 'Hello') {

  return `${greeting}, ${name}!`;

}

greet(); // 'Hello, World!'

greet('Alice'); // 'Hello, Alice!'

greet('Bob', 'Hi'); // 'Hi, Bob!'

greet(undefined, 'Hey'); // 'Hey, World!' – undefined triggers default


// Default can be any expression, even a function call

function createUser(id = generateId(), role = 'viewer') { ... }


// Arguments object (only in regular functions, not arrows)

function sum() {

  let total = 0;

  for (let i = 0; i < arguments.length; i++) {

    total += arguments[i];

  }

  return total;

}
```

```
}

// Prefer rest parameters over arguments:

function sum(...nums) {

return nums.reduce((a, b) => a + b, 0);

}
```

5.4 Closures

A closure is a function that **remembers the variables from its lexical scope** even when executed outside that scope. This is one of JavaScript's most powerful features.

```
function makeCounter(start = 0) {

let count = start; // 'count' is enclosed by inner functions

return {

increment() { return ++count; },

decrement() { return --count; },

value() { return count; },

};

}

const counter = makeCounter(10);

counter.increment(); // 11

counter.increment(); // 12

counter.decrement(); // 11

counter.value(); // 11

// Each call to makeCounter creates a NEW closure with its own 'count'
```

```

const c2 = makeCounter(); // c2.value() === 0, independent of counter

// Classic closure pitfall with var in loops

for (var i = 0; i < 3; i++) {

  setTimeout(() => console.log(i), 100); // prints 3 3 3 (shared var)

}

// Fix with let (block scope) or IIFE

for (let i = 0; i < 3; i++) {

  setTimeout(() => console.log(i), 100); // prints 0 1 2 (own binding)

}

```

5.5 IIFE — Immediately Invoked Function Expression

```

// Classic pattern to create a private scope

(function() {

  const privateData = 'secret';

  // code here doesn't pollute global scope

})();

// Arrow IIFE

(() => {

  console.log('IIFE with arrow function');

})();

// IIFE with return value

const result = (function() {

```

```
const x = 10, y = 20;

return x + y;

})(); // result === 30

// Modern note: ES Modules provide better scoping – IIFEs are less common now
```

5.6 Recursion

```
// Recursion: a function that calls itself

// Base case prevents infinite recursion

function factorial(n) {

  if (n <= 1) return 1; // base case

  return n * factorial(n - 1); // recursive case

}

factorial(5); // 120


// Fibonacci

function fib(n) {

  if (n <= 1) return n;

  return fib(n - 1) + fib(n - 2); //  $O(2^n)$  – naive, but illustrative

}


// Traversing a tree structure (real-world use case for recursion)

function sumTree(node) {

  if (!node) return 0;

  return node.value + sumTree(node.left) + sumTree(node.right);

}
```

```
}
```

■ JavaScript engines optimize **tail calls** in strict mode (TCO), but support is limited. For deep recursion, consider iteration or trampolining.

5.7 Higher-Order Functions

A higher-order function is a function that **takes a function as an argument** or **returns a function**. This is the foundation of functional programming in JS.

```
// Takes a function as argument

function repeat(n, action) {

  for (let i = 0; i < n; i++) action(i);

}

repeat(3, console.log); // 0, 1, 2


// Returns a function – function factory

function multiplier(factor) {

  return (n) => n * factor;

}

const double = multiplier(2);

const triple = multiplier(3);

double(5); // 10

triple(5); // 15


// Function composition

const compose = (f, g) => x => f(g(x));

const addOne = x => x + 1;

const square = x => x * x;

const addOneThenSquare = compose(square, addOne);
```


CHAPTER 6

Objects & Prototypes

6.1 Object Literals

```
// Creating objects

const person = {

  name: 'Alice', // property

  age: 30,

  'home city': 'NYC', // string key for non-identifier keys

  greet() { // method shorthand (ES6)

    return `Hi, I'm ${this.name}`;

  },

};

// Property access

person.name // 'Alice' – dot notation

person['age'] // 30 – bracket notation

person['home city'] // 'NYC' – bracket required for non-identifiers

const key = 'name';

person[key] // 'Alice' – dynamic key access

// ES6 Shorthand property names

const name = 'Bob';
```

```
const age = 25;

const user = { name, age }; // { name: 'Bob', age: 25 }

// Computed property names

const prop = 'score';

const obj = { [prop]: 100, [`${prop}Max`]: 200 };

// { score: 100, scoreMax: 200 }
```

6.2 The 'this' Keyword

this refers to the object that is currently executing the function. Its value depends entirely on **how** the function is called, not where it is defined.

```
// 1. Method call – this = the object before the dot

const cat = {

  name: 'Whiskers',

  speak() { return `Meow, I am ${this.name}`; }

};

cat.speak(); // 'Meow, I am Whiskers'

// 2. Regular function call – this = undefined (strict) or global

function whoAmI() { console.log(this); }

whoAmI(); // undefined in strict mode

// 3. Explicit binding: call / apply / bind

function introduce(greeting, punct) {

  return `${greeting}, ${this.name}${punct}`;

}
```

```

const me = { name: 'Dana' };

introduce.call(me, 'Hello', '!'); // call: args listed

introduce.apply(me, ['Hello', '!']); // apply: args as array

const boundFn = introduce.bind(me, 'Hey');

boundFn('.'); // 'Hey, Dana.' – bind returns a NEW function

// 4. new – this = freshly created object

function Dog(name) {

  this.name = name;

}

const rex = new Dog('Rex'); // rex.name === 'Rex'

```

6.3 The Prototype Chain

Every JavaScript object has an internal **[[Prototype]]** link to another object. Property lookup walks up this chain until it reaches **null**.

```

const animal = {

  breathe() { return 'breathing'; }

};

const dog = Object.create(animal); // dog's prototype = animal

dog.bark = function() { return 'woof'; };

dog.bark(); // 'woof' – own property

dog.breathe(); // 'breathing' – found on prototype (animal)

// Check prototype chain

```

```
Object.getPrototypeOf(dog) === animal; // true

dog instanceof Object; // true


// hasOwnProperty – only own, not inherited

dog.hasOwnProperty('bark'); // true

dog.hasOwnProperty('breathe'); // false


// All objects ultimately trace back to Object.prototype

// Object.prototype.__proto__ === null (end of chain)
```

6.4 Constructor Functions and new

```
function Person(name, age) {

  this.name = name;

  this.age = age;

}


// Methods on prototype – shared across all instances (memory efficient)

Person.prototype.greet = function() {

  return `Hi, I'm ${this.name}`;

};


const alice = new Person('Alice', 30);

const bob = new Person('Bob', 25);


// What 'new' does:

// 1. Creates a new empty object {}
```

```
// 2. Sets its [[Prototype]] to Person.prototype

// 3. Executes the constructor with 'this' = new object

// 4. Returns the new object (unless constructor returns an object)

alice.greet(); // 'Hi, I'm Alice'

alice instanceof Person; // true
```

6.5 Object Methods Reference

```
const obj = { a: 1, b: 2, c: 3 };

Object.keys(obj) // ['a', 'b', 'c'] – own enumerable keys

Object.values(obj) // [1, 2, 3] – own enumerable values

Object.entries(obj) // [['a',1],['b',2],['c',3]]

// Shallow clone

const clone1 = Object.assign({}, obj);

const clone2 = { ...obj }; // spread (preferred)

// Merge objects

const merged = { ...obj, d: 4, a: 99 }; // later keys win: a=99

// Freeze – prevent mutations

const frozen = Object.freeze({ x: 1 });

frozen.x = 99; // silently fails (throws in strict mode)

// Create with null prototype (no inherited methods)

const bare = Object.create(null);
```

```
// Property descriptors

Object.defineProperty(obj, 'secret', {

  value: 42,

  writable: false,

  enumerable: false,

  configurable: false,

});
```

6.6 Destructuring

```
// Object destructuring

const { name, age } = person;

// Rename while destructuring

const { name: personName, age: personAge } = person;

// Default values

const { role = 'guest', name: n } = user;

// Nested destructuring

const { address: { city, zip } } = user;

// Function parameter destructuring

function display({ name, age = 0 }) {

  console.log(name, age);

}
```


CHAPTER 7

Arrays — Complete Guide

7.1 Creating Arrays

```
const arr1 = [1, 2, 3]; // array literal (preferred)

const arr2 = new Array(3); // [empty × 3] – 3 empty slots

const arr3 = new Array(1, 2, 3); // [1, 2, 3]

const arr4 = Array.from('hello'); // ['h','e','l','l','o']

const arr5 = Array.from({length:5}, (_,i) => i*2); // [0,2,4,6,8]

const arr6 = Array.of(1, 2, 3); // [1, 2, 3]

const arr7 = [...someIterable]; // spread


// Basic operations

arr1.length // 3

arr1[0] // 1 (zero-indexed)

arr1[arr1.length - 1] // 3 (last element)

arr1.at(-1) // 3 (ES2022 – negative indexing)
```

7.2 Mutating Methods (modify the original array)

```
const a = [1, 2, 3];


a.push(4) // adds to end → returns new length → a=[1,2,3,4]

a.pop() // removes last → returns element → a=[1,2,3]

a.unshift(0) // adds to front → returns new length → a=[0,1,2,3]

a.shift() // removes first → returns element → a=[1,2,3]
```

```

// splice(startIndex, deleteCount, ...itemsToInsert)

a.splice(1, 0, 'x') // insert 'x' at index 1, delete 0

a.splice(1, 2) // remove 2 elements starting at index 1

a.splice(0, 1, 'new') // replace index 0 with 'new'


a.reverse() // reverses in place

a.sort() // sorts in place – default: lexicographic (!)


// Numeric sort – MUST provide comparator

[10, 1, 21, 2].sort((a, b) => a - b) // [1, 2, 10, 21] ascending

[10, 1, 21, 2].sort((a, b) => b - a) // [21, 10, 2, 1] descending


a.fill(0, 2, 4) // fill with 0 from index 2 to 4 (exclusive)

a.copyWithin(0, 3, 5) // copy elements 3..4 to position 0

```

7.3 Non-Mutating Methods (return new array or value)

```

const nums = [1, 2, 3, 4, 5];


// slice(start, end) – shallow copy from start to end (exclusive)

nums.slice(1, 3) // [2, 3]

nums.slice(-2) // [4, 5]

nums.slice() // full shallow copy


// concat – merge arrays

[1, 2].concat([3, 4], [5]) // [1, 2, 3, 4, 5]

[1, 2, ...[3, 4]] // same with spread (preferred)

```

```
// flat / flatMap

[1, [2, [3]]].flat() // [1, 2, [3]] – 1 level

[1, [2, [3]]].flat(Infinity) // [1, 2, 3] – all levels

[1,2,3].flatMap(x => [x, x*2]) // [1,2,2,4,3,6]


// join – convert to string

[1,2,3].join('-') // '1-2-3'

[1,2,3].join('') // '123'

[1,2,3].join() // '1,2,3' (default comma)
```

7.4 Iteration Methods — The Core Toolkit

These are the most important array methods. Master these and you'll write clean, expressive JavaScript.

```
const nums = [1, 2, 3, 4, 5];


// forEach – execute side effects for each element (returns undefined)

nums.forEach((n, index) => console.log(index, n));


// map – transform each element, returns new array of same length

nums.map(n => n * 2) // [2, 4, 6, 8, 10]

nums.map(n => ({ value: n })) // [{value:1}, {value:2}, ...]


// filter – keep elements where callback returns true

nums.filter(n => n % 2 === 0) // [2, 4]

nums.filter(Boolean) // removes falsy values


// reduce – accumulate to a single value
```


■ Chaining: **array.filter(...).map(...).reduce(...)** is idiomatic JavaScript. Each method returns an array (except **reduce/find/some/every**), enabling chains.

7.5 Array Searching & Testing

```
// Array.isArray – the correct way to check for arrays

Array.isArray([1,2,3]) // true

Array.isArray('string') // false

typeof [] === 'object' // true – can't use typeof for arrays


// findLast / findLastIndex (ES2023)

[1,2,3,4].findLast(n => n % 2 === 0) // 4

[1,2,3,4].findLastIndex(n => n % 2 === 0) // 3


// toSorted / toReversed / toSpliced / with (ES2023 – non-mutating!)

const orig = [3,1,2];

const sorted = orig.toSorted((a,b) => a-b); // [1,2,3]

// orig is still [3,1,2]
```

CHAPTER 8

ES6+ Modern JavaScript

8.1 Classes (ES6)

ES6 classes are **syntactic sugar** over prototype-based inheritance. They do not introduce a new object model — they make the existing prototype system easier to write and read.

```
class Animal {  
  
  #sound; // Private field (ES2022) – # prefix  
  
  constructor(name, sound) {  
  
    this.name = name;  
  
    this.#sound = sound;  
  
  }  
  
  speak() { // Instance method (on prototype)  
  
    return `${this.name} says ${this.#sound}`;  
  
  }  
  
  static create(name, sound) { // Static method – called on class  
  
    return new Animal(name, sound);  
  
  }  
  
  get info() { // Getter  
  
    return `[Animal: ${this.name}]`;  
  
  }  
  
}
```

8.2 ES Modules (import / export)

```
// ■■ math.js ██████████  
  
export const PI = 3.14159;  
  
export function add(a, b) { return a + b; }
```

- ES Modules are always in strict mode and have their own scope. Use them in browsers with `<script type="module">` or in Node.js with `.mjs` extension or `"type":"module"` in package.json.

```

const html = `

<div>

<p>${name}</p>

</div>

`;

// Tagged templates – process template with a function

function highlight(strings, ...values) {

return strings.reduce((result, str, i) =>

result + str + (values[i] ? `<b>${values[i]}</b>` : ''),

'');

}

const msg = highlight`Hello ${name}, you have ${5} messages`;

// 'Hello <b>World</b>, you have <b>5</b> messages'

// String.raw – tagged template that ignores escape sequences

String.raw`C:\Users\Alice` // 'C:\Users\Alice' (no escape processing)

```

8.4 Symbols

```

// Symbol – unique primitive, often used as object keys

const id = Symbol('id'); // 'id' is just a description

const id2 = Symbol('id');

id === id2; // false – every Symbol is unique

const user = {

```

```

name: 'Alice',

[id]: 123456, // Symbol as key – not in for..in, JSON.stringify

};

// Well-known Symbols – hooks into JS engine behavior

class MyArray {

  [Symbol.iterator]() { // makes instances iterable

    let i = 0, data = [1, 2, 3];

    return { next: () => ({ value: data[i++], done: i > data.length }) };

  }

}

for (const n of new MyArray()) console.log(n); // 1 2 3

// Other well-known Symbols: Symbol.toPrimitive, Symbol.hasInstance,

// Symbol.species, Symbol.toStringTag, Symbol.asyncIterator

```

8.5 Map and Set

```

// Map – key-value pairs where keys can be ANY type

const map = new Map();

map.set('key', 'value');

map.set(42, 'number key');

map.set({}, 'object key');

map.get('key') // 'value'

map.has(42) // true

```



```
const difference = new Set([...a].filter(x => !b.has(x)));
```

8.6 Iterators and Generators

```
// Iterator protocol – object with next() returning { value, done }

function range(start, end) {

  let current = start;

  return {

    [Symbol.iterator]() { return this; },

    next() {

      return current <= end

        ? { value: current++, done: false }

        : { value: undefined, done: true };

    }

  };

}

for (const n of range(1, 5)) console.log(n); // 1 2 3 4 5

// Generator function – function* – pauses at each yield

function* counter(start = 0) {

  while (true) {

    const reset = yield start++; // yield sends value out, receives value in

    if (reset) start = 0;

  }

}
```


CHAPTER 9

Asynchronous JavaScript

9.1 The Problem: Single Thread + I/O

JavaScript runs on a **single thread** — only one piece of code executes at a time. Yet, it needs to handle I/O operations (network requests, timers, file reads) that can take milliseconds to seconds without freezing.

The solution is an **asynchronous, non-blocking I/O model** powered by the **event loop** and **callback queue** (detailed in Ch. 12). The key insight: JavaScript doesn't wait for I/O — it registers a callback and moves on.

9.2 Callbacks (legacy pattern)

```
// Callbacks were the original async pattern

setTimeout(() => console.log('1 second later'), 1000);

// Node-style callbacks: (error, result) – 'error-first convention'

fs.readFile('data.txt', 'utf8', (err, data) => {

  if (err) { console.error(err); return; }

  console.log(data);

});

// Callback Hell – deeply nested, hard to read and maintain

getUser(id, (err, user) => {

  if (err) handle(err);

  getPosts(user.id, (err, posts) => {

    if (err) handle(err);

    getComments(posts[0].id, (err, comments) => {

      // ... the pyramid of doom
```

```
});
```

```
});
```

```
});
```

9.3 Promises

A **Promise** is an object representing the eventual completion or failure of an async operation. It has three states: **pending**, **fulfilled**, or **rejected**.

```
// Creating a Promise

const p = new Promise((resolve, reject) => {

  setTimeout(() => {

    const success = Math.random() > 0.5;

    if (success) resolve('Data received');

    else reject(new Error('Network failure'));

  }, 1000);

});

// Consuming a Promise

p.then(data => console.log(data)) // called if resolved

.catch(err => console.error(err)) // called if rejected

.finally(() => console.log('done')); // always called

// then() is chainable – each then returns a new Promise

fetch('/api/users') // returns a Promise

.then(res => res.json()) // parse JSON – also async!

.then(data => console.log(data))

.catch(err => console.error(err));
```

```
// Promisifying a callback-based function

function readFilePromise(path) {

  return new Promise((resolve, reject) => {

    fs.readFile(path, 'utf8', (err, data) =>

      err ? reject(err) : resolve(data)

    );

  });

}
```

9.4 Promise Combinators

```
const p1 = fetch('/api/users');

const p2 = fetch('/api/posts');

const p3 = fetch('/api/tags');


// Promise.all – waits for ALL, rejects if ANY fails

const [users, posts, tags] = await Promise.all([p1, p2, p3]);


// Promise.allSettled – waits for ALL, never rejects

// Returns array of { status: 'fulfilled'|'rejected', value|reason }

const results = await Promise.allSettled([p1, p2, p3]);

results.forEach(r => {

  if (r.status === 'fulfilled') console.log(r.value);

  else console.error(r.reason);

});
```

```
// Promise.race – resolves/rejects with the FIRST settled promise

const timeout = new Promise( (_,r) => setTimeout(() => r(new Error('timeout')),5000));

const result = await Promise.race([fetchData(), timeout]);

// Promise.any – resolves with FIRST fulfilled, rejects only if ALL fail

const fastest = await Promise.any([mirror1(), mirror2(), mirror3()]);
```

9.5 async / await (ES2017)

async/await is **syntactic sugar over Promises**. It makes asynchronous code look and behave like synchronous code, dramatically improving readability.

```
// async function always returns a Promise

async function fetchUser(id) {

  // await pauses execution until the Promise settles

  const response = await fetch(`/api/users/${id}`);

  if (!response.ok) {

    throw new Error(`HTTP error: ${response.status}`);

  }

  const user = await response.json();

  return user; // auto-wrapped in Promise.resolve(user)

}

// Error handling with try/catch

async function loadProfile(id) {

  try {

    const user = await fetchUser(id);
```

```

const posts = await fetchPosts(user.id); // sequential

return { user, posts };

} catch (err) {

console.error('Failed:', err.message);

return null;

} finally {

hideLoading();

}

}

// Parallel with async/await – use Promise.all

async function loadAll(id) {

const [user, settings] = await Promise.all([

fetchUser(id),

fetchSettings(id),

]);

return { user, settings };

}

// Top-level await (in ES Modules)

const config = await loadConfig();

```

9.6 Async Iteration (ES2018)

```

// for await...of – iterate over async iterables

async function processStream(stream) {

```


CHAPTER 10

Error Handling & Debugging

10.1 try / catch / finally

```
try {

  // Code that might throw

  const data = JSON.parse(invalidJSON); // throws SyntaxError

  riskyOperation();

} catch (err) {

  // err is the thrown value – usually an Error object

  console.error(err.name); // 'SyntaxError'

  console.error(err.message); // description

  console.error(err.stack); // stack trace

} finally {

  // ALWAYS executes – even if try returns or catch throws

  cleanup();

}

// Catching specific error types

try {

  JSON.parse(badInput);

} catch (err) {

  if (err instanceof SyntaxError) {
```

```
console.error('Invalid JSON');

} else {

  throw err; // re-throw unknown errors

}

}
```

10.2 Error Types

- **Error** — base type
- **SyntaxError** — invalid syntax (parse time)
- **ReferenceError** — accessing undefined variable
- **TypeError** — wrong type operation (call non-function, property of null)
- **RangeError** — value out of valid range (invalid array length)
- **URIError** — malformed URI
- **EvalError** — eval() errors (rare)

```
null.property // TypeError

undeclaredVar // ReferenceError

new Array(-1) // RangeError
```

10.3 Custom Errors

```
class AppError extends Error {

  constructor(message, code, data = null) {

    super(message); // sets this.message

    this.name = 'AppError'; // override name

    this.code = code;

    this.data = data;

    // Fix prototype chain for instanceof

    Object.setPrototypeOf(this, new.target.prototype);

  }

}
```

```

}

class NotFoundError extends AppError {

  constructor(resource, id) {

    super(`${resource} with id ${id} not found`, 'NOT_FOUND');

    this.name = 'NotFoundError';

  }

}

throw new NotFoundError('User', 42);

// Catch by type

try { ... }

catch (err) {

  if (err instanceof NotFoundError) { handle404(err); }

  else if (err instanceof AppError) { handleApp(err); }

  else throw err;

}

```

10.4 The throw Statement

```

// throw any value – but always throw Error objects

throw 'some string'; // valid but bad practice

throw 42; // valid but bad practice

throw new Error('message'); // correct

throw new TypeError('Expected a number');

```


CHAPTER 11

Advanced Patterns & Concepts

11.1 Scope in Depth

Scope determines the visibility of variables. JavaScript has **global scope**, **function scope**, and **block scope** (ES6+).

```
// Lexical scope – scope is determined by WHERE code is written

const outer = 'I am outer';

function inner() {

  console.log(outer); // outer is accessible – lexical lookup

  const local = 'inner only';

}

// Block scope with let/const

{

  let blockVar = 'only here';

  const blockConst = 'also only here';

}

// blockVar and blockConst are not accessible here

// Scope chain – JS looks up the chain until it finds the variable

const x = 'global';

function outer() {

  const x = 'outer';

  function inner() {
```

```

const x = 'inner';

console.log(x); // 'inner' – own scope wins

}

inner();

console.log(x); // 'outer'

}

```

11.2 Hoisting

```

// Function declarations – fully hoisted

console.log(greet('JS')); // Works! 'Hello, JS'

function greet(name) { return `Hello, ${name}`; }

// var declarations – hoisted as undefined

console.log(x); // undefined (not an error!)

var x = 5;

// Equivalent to: var x; console.log(x); x = 5;

// let / const – hoisted but in Temporal Dead Zone (TDZ)

console.log(y); // ReferenceError: Cannot access 'y' before initialization

let y = 10;

// Class declarations – also TDZ (unlike function declarations)

const obj = new MyClass(); // ReferenceError

class MyClass {}

```

11.3 Currying

Currying transforms a function that takes multiple arguments into a sequence of functions each taking one argument.


```
add3(1, 2, 3); // 6
```

11.4 Memoization

```
// Cache expensive computation results

function memoize(fn) {

  const cache = new Map();

  return function(...args) {

    const key = JSON.stringify(args);

    if (cache.has(key)) {

      return cache.get(key); // cache hit

    }

    const result = fn.apply(this, args);

    cache.set(key, result);

    return result;

  };

}

const memoFib = memoize(function fib(n) {

  if (n <= 1) return n;

  return memoFib(n - 1) + memoFib(n - 2); // O(n) now!

});

memoFib(40); // fast!
```

11.5 Proxy and Reflect

Proxy lets you intercept and customize fundamental object operations. **Reflect** provides a clean API to perform those operations.

11.6 Common Design Patterns

Module Pattern

```
const BankAccount = (() => {

  let balance = 0; // private

  return {

    deposit(amt) { balance += amt; },

    withdraw(amt) { if (amt <= balance) balance -= amt; },

    getBalance() { return balance; },

  };

})();
```

Observer Pattern

```
class EventEmitter {

  #listeners = new Map();

  on(event, fn) {

    if (!this.#listeners.has(event)) this.#listeners.set(event, []);

    this.#listeners.get(event).push(fn);

    return () => this.off(event, fn); // returns unsubscribe fn

  }

  off(event, fn) {

    const fns = this.#listeners.get(event) ?? [];

    this.#listeners.set(event, fns.filter(f => f !== fn));

  }

}
```


CHAPTER 12

The JavaScript Runtime

12.1 Memory Model

JavaScript has two memory regions: the **Stack** and the **Heap**.

- **Stack:** Stores primitive values, function call frames, and references. Fast, fixed size, LIFO order. Each function call pushes a frame; return pops it.
- **Heap:** Stores objects, arrays, closures. Dynamic size. Variables hold *references* (pointers) to heap locations.

```
let a = 5; // primitive – value 5 stored on stack

let b = a; // primitive copy – b gets its own 5

a = 10; // changing a doesn't affect b

let obj1 = { x: 1 }; // object stored on heap, ref on stack

let obj2 = obj1; // obj2 holds SAME reference – points to same heap object

obj1.x = 99;

console.log(obj2.x); // 99 – they share the same object
```

12.2 Garbage Collection

JavaScript uses **mark-and-sweep** garbage collection. The GC periodically marks all reachable objects (from roots: global, stack), then sweeps/frees unreachable ones.

```
// Memory leak example – unintentional reference keeps object alive

const cache = {};

function process(key, data) {

  cache[key] = data; // grows forever if keys are unique

}

// Fix: use WeakMap (doesn't prevent GC of keys)
```

```

const weakCache = new WeakMap();

function processWeak(objKey, data) {

  weakCache.set(objKey, data); // GC can collect when objKey is gone

}

// Common leak sources:

// - Forgotten timers/intervals

// - Detached DOM nodes stored in variables

// - Closures capturing large objects unnecessarily

// - Global variables

```

12.3 The Event Loop — Deep Dive

The event loop is what enables JavaScript's non-blocking behavior despite being single-threaded. Understanding it is essential for writing correct async code.

- **Call Stack:** Where currently executing functions live. Synchronous. LIFO.
- **Web APIs / Node APIs:** Off-thread environment for async ops (setTimeout, fetch, I/O).
- **Macrotask Queue (Task Queue):** setTimeout, setInterval, I/O callbacks, UI events.
- **Microtask Queue:** Promise callbacks (.then/.catch), queueMicrotask(), MutationObserver.

```

// Event loop algorithm (simplified):

// 1. Execute all synchronous code (drain the call stack)

// 2. Drain the ENTIRE microtask queue

// 3. Take ONE task from macrotask queue, execute it

// 4. Drain microtask queue again

// 5. Render (browser) if needed

// 6. Go to step 3

console.log('1 - sync');

```

```

setTimeout(() => console.log('4 - macrotask'), 0);

Promise.resolve()

.then(() => console.log('3 - microtask 1'))

.then(() => console.log('3b- microtask 2 (chained)'));

console.log('2 - sync');

// Output: 1, 2, 3, 3b, 4

// Even though setTimeout has 0ms delay, microtasks run first!

```

■ *Microtasks always drain completely before the next macrotask runs. Scheduling too many microtasks in a loop can starve the macrotask queue and block rendering.*

12.4 Performance Tips

- **Avoid blocking the main thread:** Move heavy computation to Web Workers.
- **Debounce and throttle:** Limit how often expensive callbacks (scroll, resize, input) fire.
- **Use const > let > var** for engine optimization hints.
- **Avoid repeated DOM queries:** Cache DOM references.
- **Prefer for loops over forEach** for hot paths (minor, but measurable at scale).
- **Avoid memory leaks:** Clear intervals, remove event listeners when done.
- **Use lazy loading:** Dynamic **import()** for code splitting.

```

// Debounce – wait until N ms after LAST call

function debounce(fn, delay) {

  let timer;

  return (...args) => {

    clearTimeout(timer);

    timer = setTimeout(() => fn(...args), delay);

  };

}

```

```
// Throttle – allow at most one call per N ms

function throttle(fn, interval) {

  let last = 0;

  return (...args) => {

    const now = Date.now();

    if (now - last >= interval) {

      last = now;

      fn(...args);

    }

  };

}

const onScroll = throttle(() => updateNavbar(), 100);

window.addEventListener('scroll', onScroll);
```

12.5 What to Study Next

You now have a complete foundation. Here are the natural next steps:

- **DOM & Browser APIs:** document, events, fetch, LocalStorage, Web Workers, WebSockets
- **Node.js:** File system, HTTP, streams, npm, Express/Fastify
- **TypeScript:** Static typing layer over JavaScript — industry standard
- **Testing:** Jest, Vitest, Testing Library — unit & integration testing
- **Frameworks:** React, Vue, Svelte — built entirely on the JS you now know
- **Tooling:** Vite, esbuild, Webpack — bundlers, transpilers
- **Patterns:** Functional programming (Ramda), reactive (RxJS), state machines (XState)

You've completed the course.

JavaScript mastery is not about memorizing syntax — it's about building mental models.

You now understand the engine, the language, and the patterns.

MIT JavaScript Course · ES3 through ES2024